

Detectar un ataque con Suricata

Del paquete a la regla.

Roy Sánchez Prato | Alex Parejas, Seguridad y alta disponibilidad | 2º ASIR Caja Mágica | 28/09/2025

Índice

1) Preparación de las máquinas virtuales.....3
2) Instalación de Suricata en el Servidor (Instalar el IDS y el servicio inseguro)..... 8
3) Máquina víctima (Configurar Suricata).....9
4) Atacar y observar (Provocar el ataque y ver qué detecta Suricata de fábrica)..... 13
5) Contramedida.....19
Cuestionarios.....22

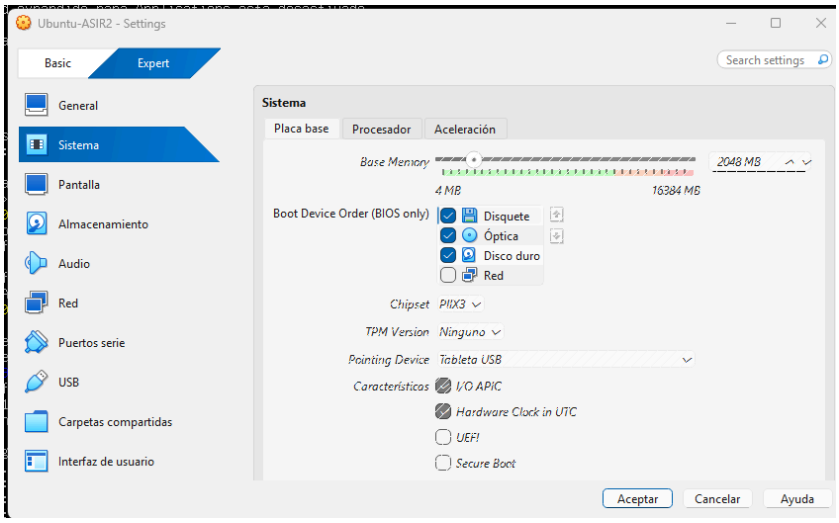
Introducción

En esta práctica que se propone para la asignatura, el objetivo es poner en práctica la teoría en un entorno real de sistemas y comprobar cómo funciona un Sistema de Detección de Intrusiones (IDS). Siguiendo las directrices del guión, he desplegado Suricata en un servidor Ubuntu y he configurado un servicio vulnerable FTP a propósito. Esto me ha permitido simular y analizar todo el ciclo de un ataque: desde la fase inicial de reconocimiento mediante escaneo de puertos, hasta la interceptación de credenciales y su detección mediante la creación de reglas personalizadas. Finalmente, el laboratorio concluye con la fortificación del servidor mediante un cortafuegos, cerrando la brecha y reduciendo drásticamente la superficie de ataque.

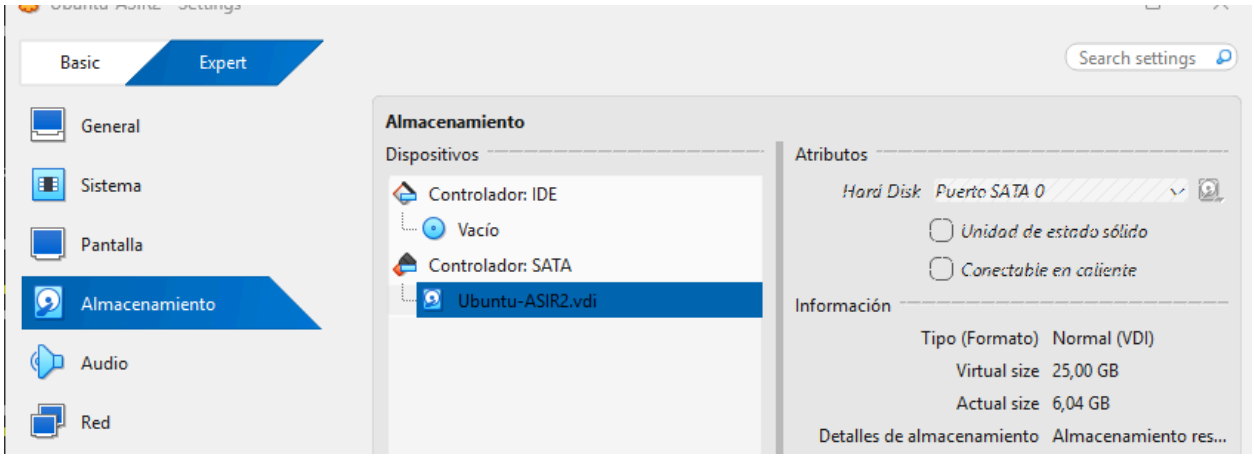
1) Preparación de las máquinas virtuales.

Para esta práctica será necesaria 2 máquinas virtuales Ubuntu. Una de ellas será Ubuntu Server y la otra será Ubuntu Usuario.

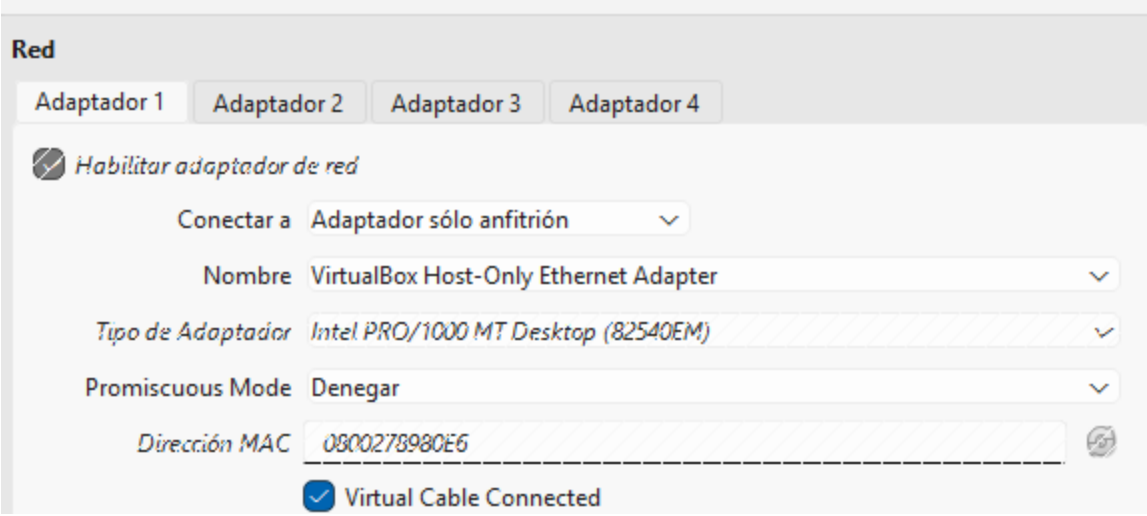
1.1 Instalación de Ubuntu Server.



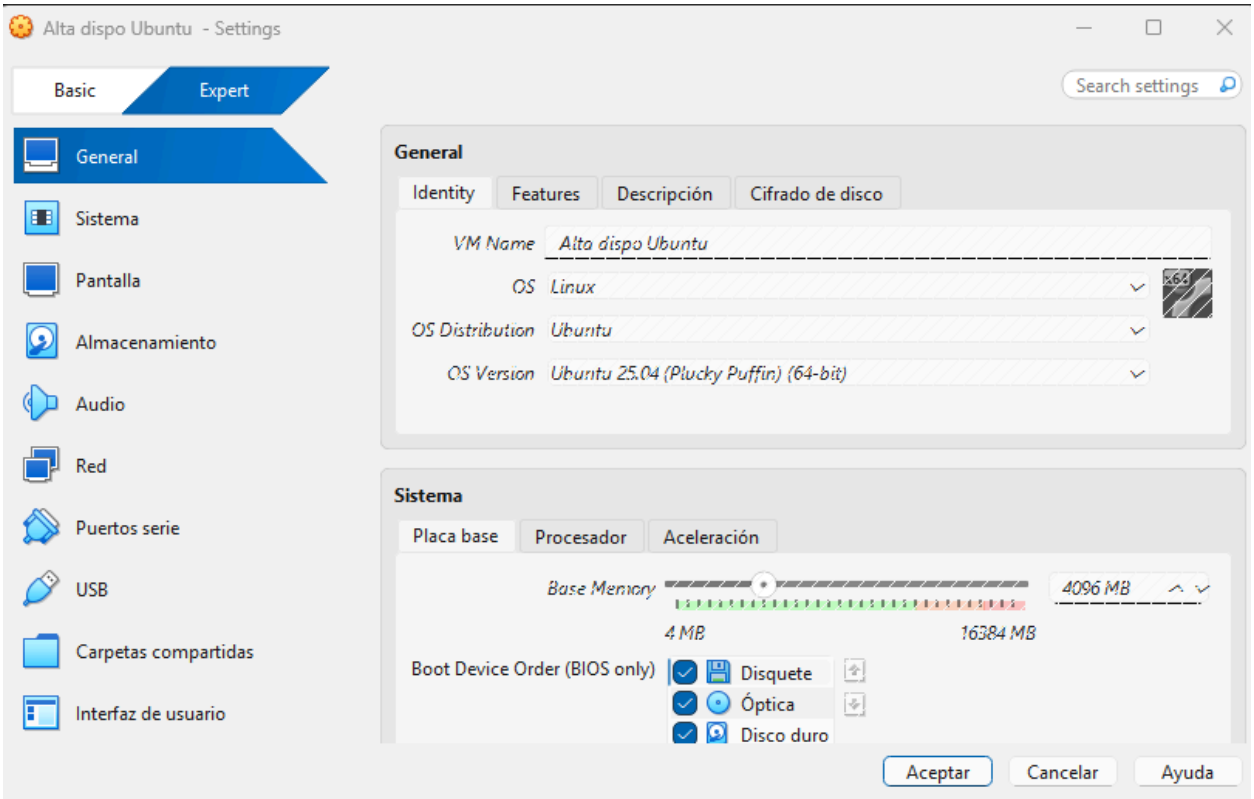
Le ponemos 2 GB de RAM y de almacenamiento 25 GB



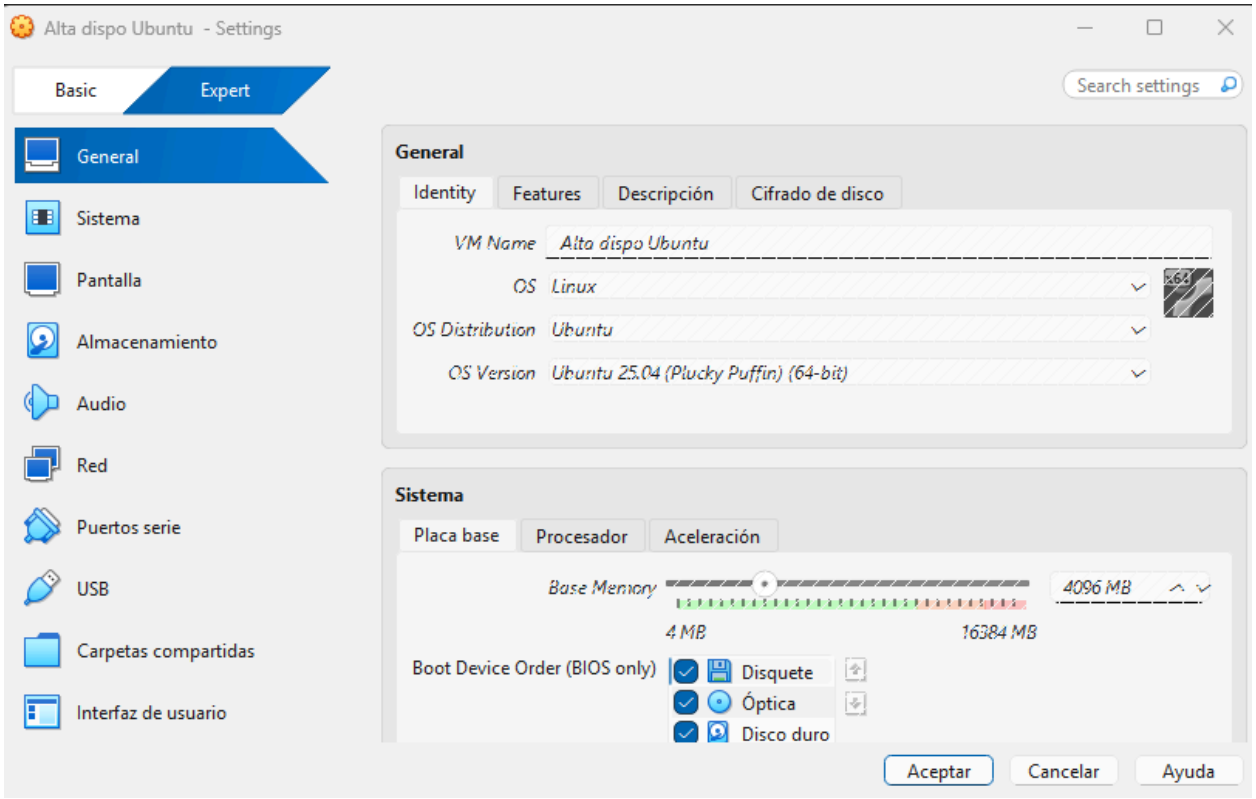
La red es muy importante ponerla en Adaptador sólo anfitrión para esta práctica:



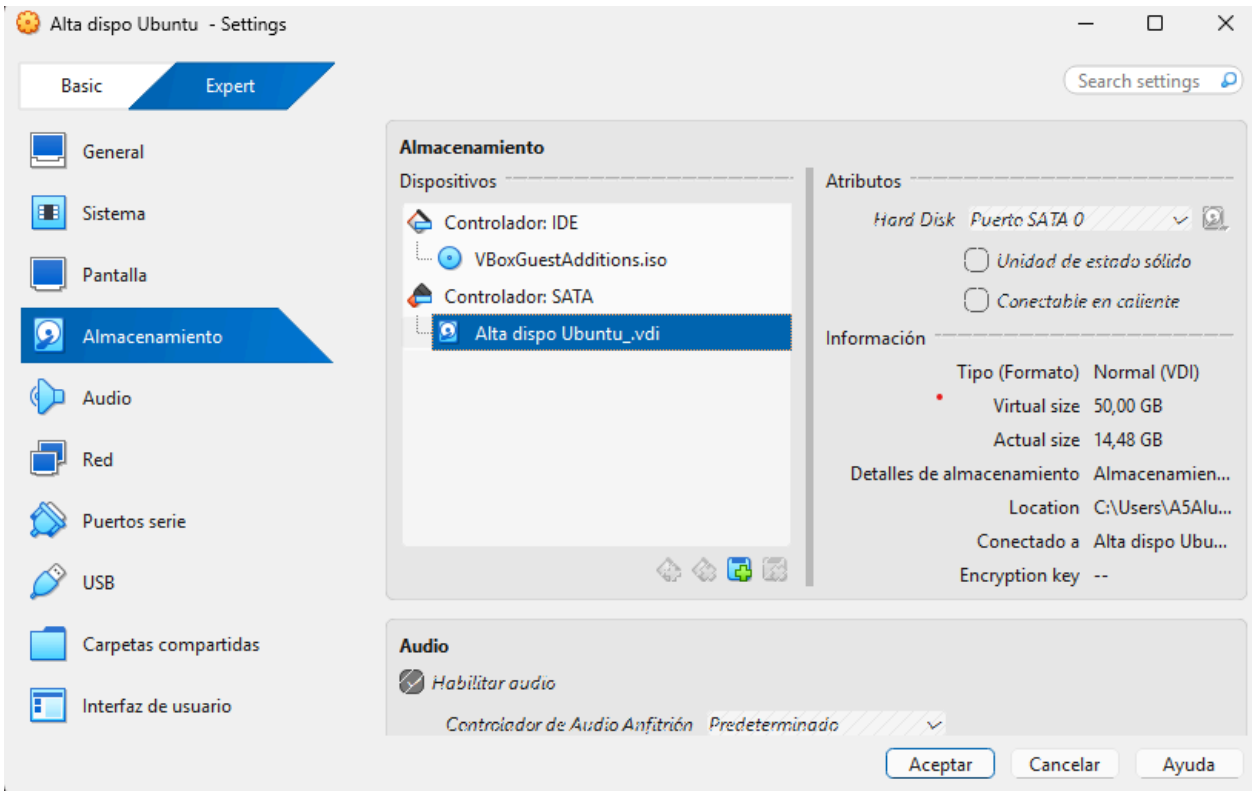
1.2 Instalación de Ubuntu Usuario



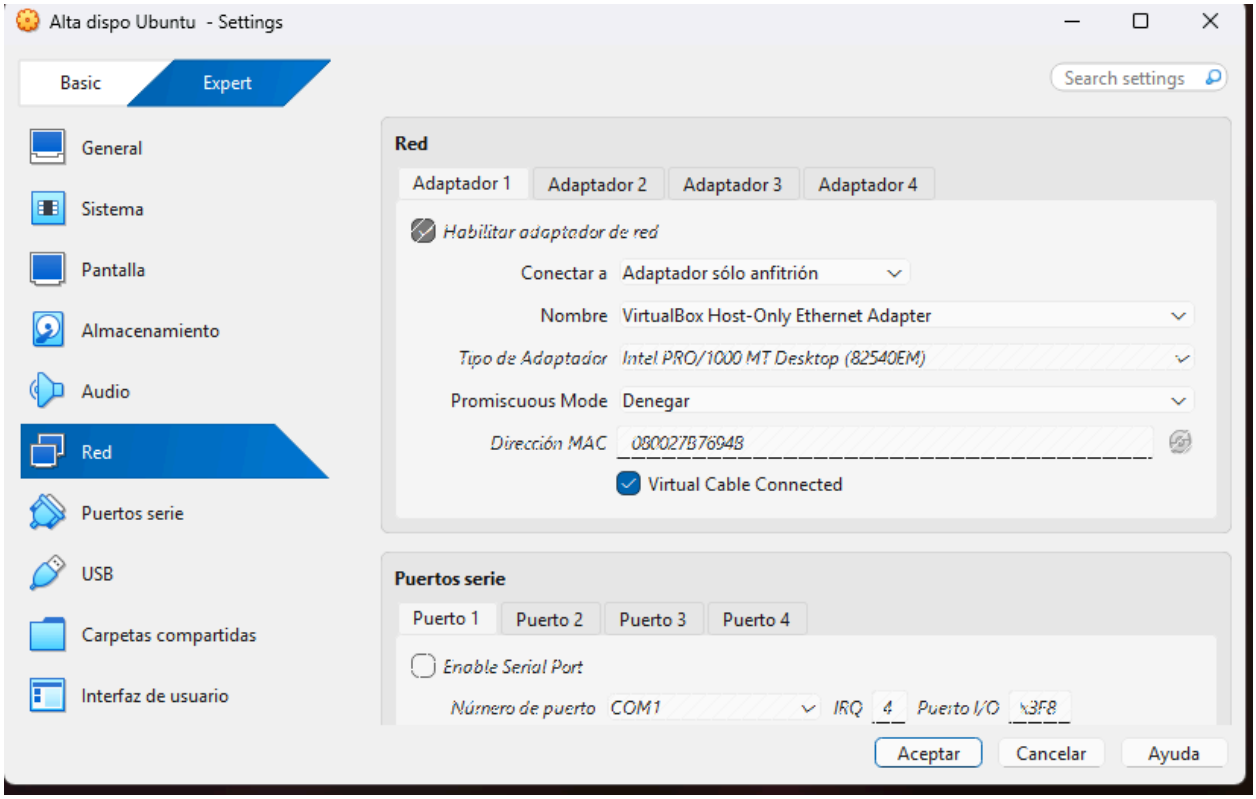
Le ponemos 4 GB de RAM



Para disco duro le hemos puesto

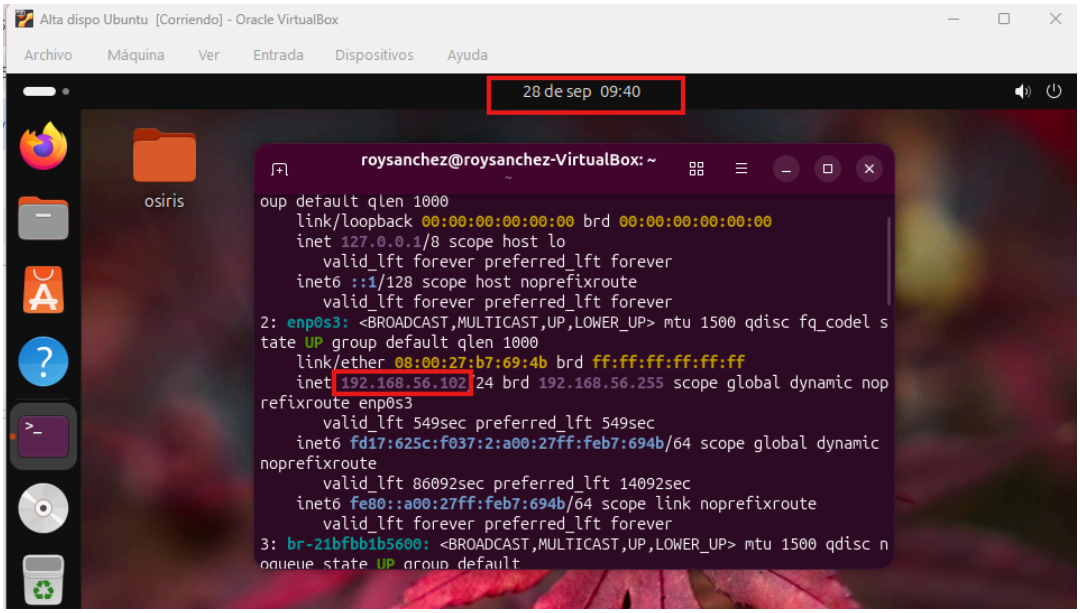


A esta máquina también le ponemos la red en adaptador solo anfitrión.

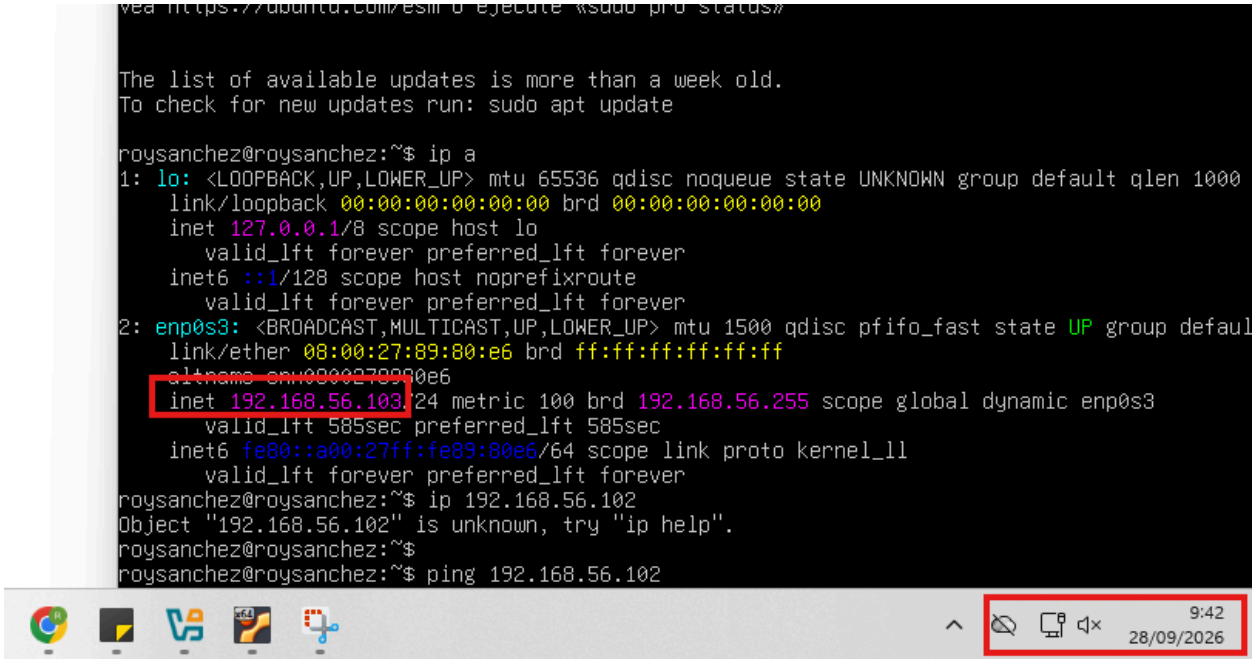


1.3 Prueba de conectividad entre máquinas.

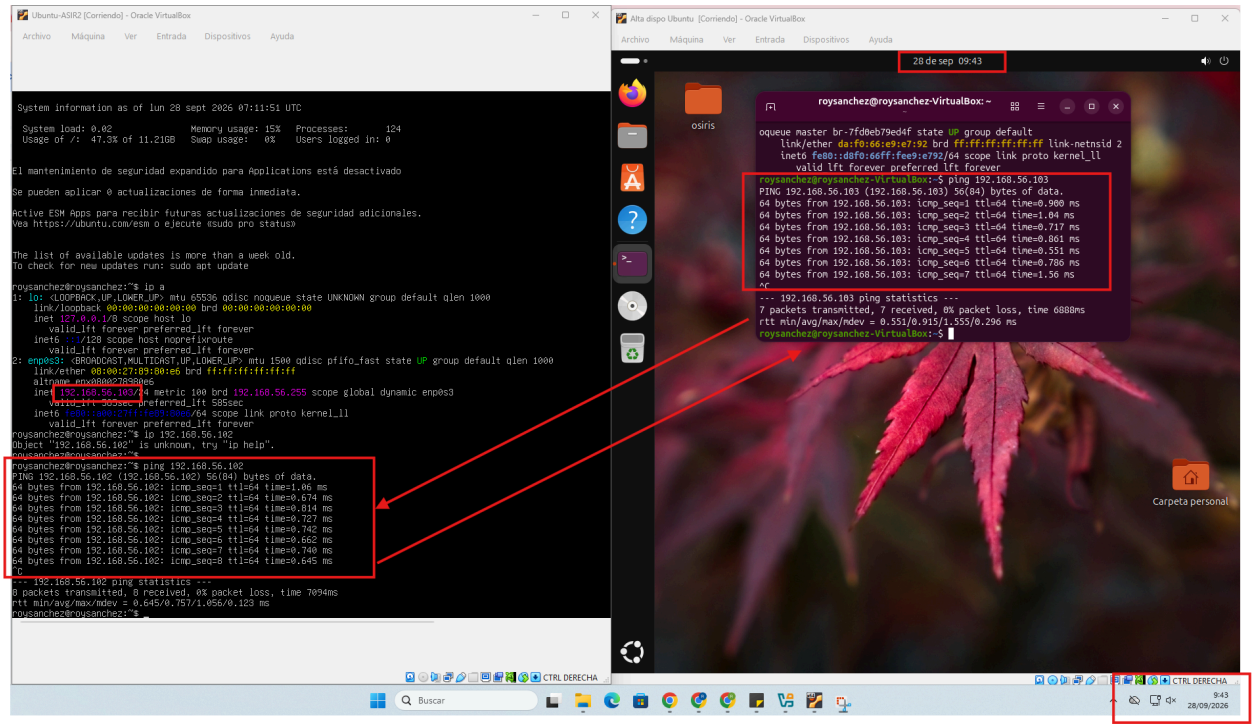
Verificación de IP a de la máquina Ubuntu usuario es 192.168.56.102



Verificación de la IP del Ubuntu de servidor es 192.168.56.103.



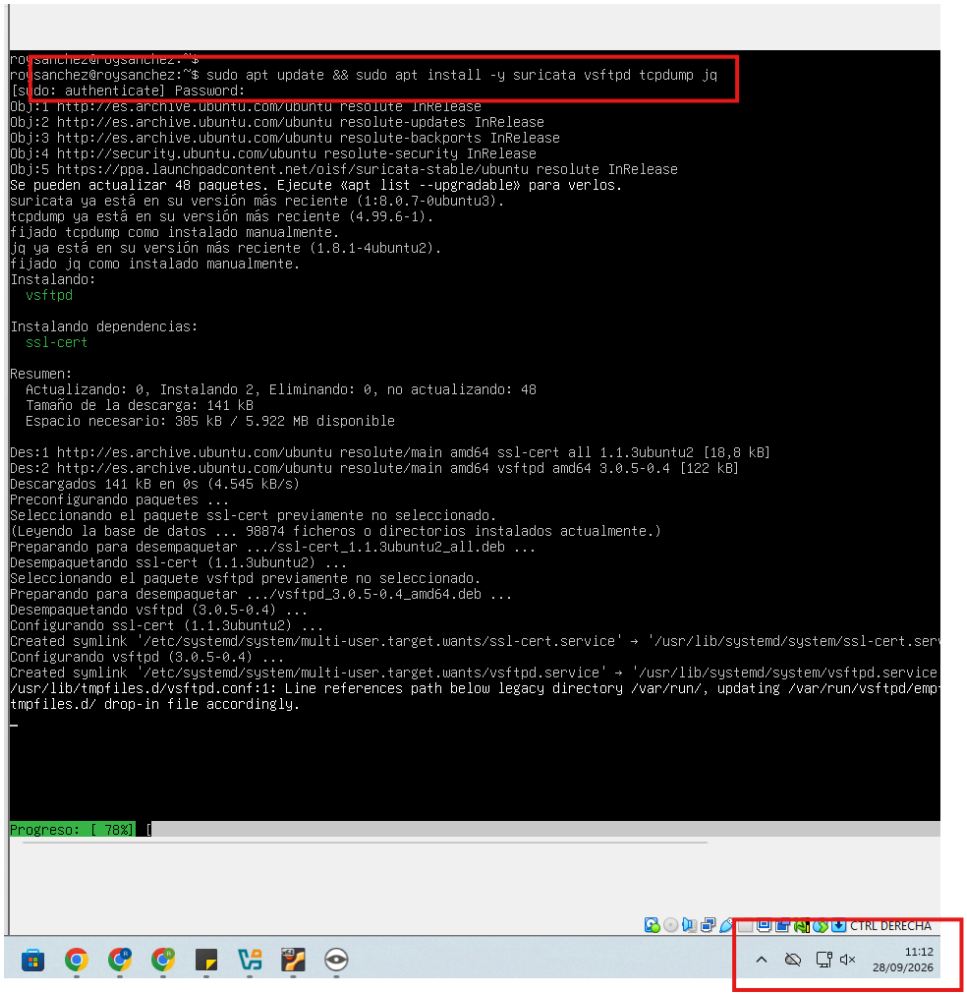
Prueba de conexión entre máquinas.



2) Instalación de Suricata en el Servidor (Instalar el IDS y el servicio inseguro)

- **Instalar el software necesario:**

sudo apt update && sudo apt install -y suricata vsftpd tcpdump jq



- **Crear el usuario de prueba:**

sudo adduser alumno

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
roysanchez@roysanchez:~$ sudo adduser alumno
New password:
Retype new password:
passwd: contraseña actualizada correctamente
Cambiando la información de usuario para alumno
Introduzca el nuevo valor, o pulse INTRO para usar el valor predeterminado
        Nombre completo []: Roy
        Número de habitación []: 1
        Teléfono del trabajo []: 1
        Teléfono de casa []: 1
        Otro []: 1
Is the information correct? [Y/n] Y
roysanchez@roysanchez:~$
```

anotamos la IP con

ip a

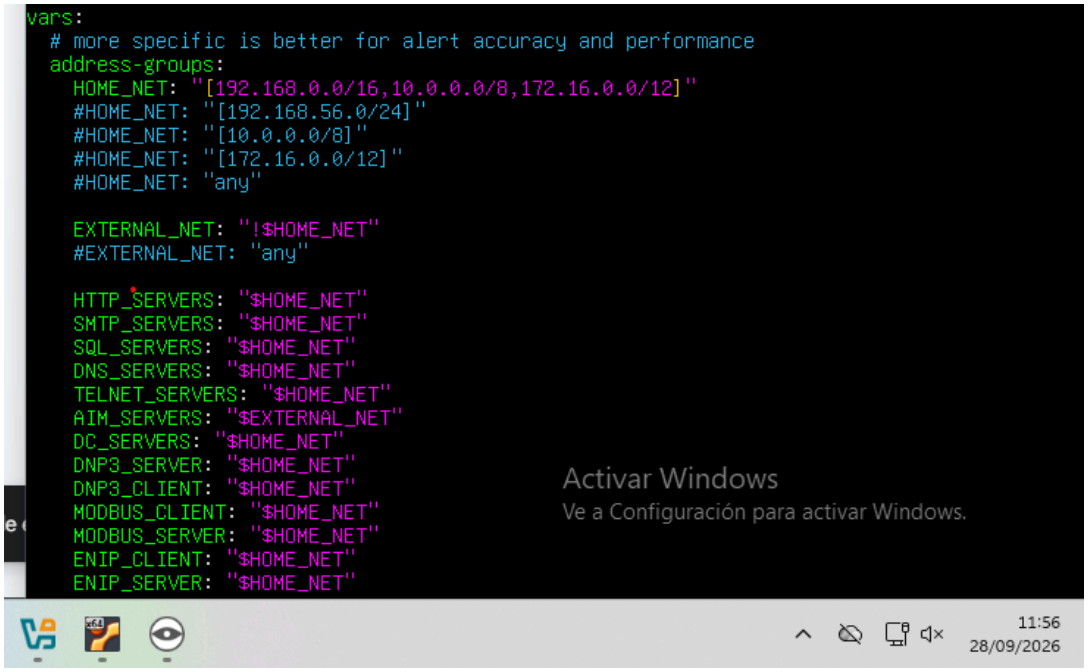
```
Y: command not found
roysanchez@roysanchez:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:89:80:e6 brd ff:ff:ff:ff:ff:ff
    altname enx0800278980e6
    inet 192.168.56.103/24 metric 100 brd 192.168.56.255 scope global dynamic enp0s3
       valid_lft 509sec preferred_lft 509sec
    inet6 fe80::a00:27ff:fe89:80e6/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:54:36:e7 brd ff:ff:ff:ff:ff:ff
    altname enx0800275436e7
    inet 10.0.3.15/24 metric 1024 brd 10.0.3.255 scope global dynamic enp0s8
       valid_lft 81506sec preferred_lft 81506sec
    inet6 fd17:625c:f037:3:a00:27ff:fe54:36e7/64 scope global dynamic mngtmpaddr noprefixroute
       valid_lft 86049sec preferred_lft 14049sec
    inet6 fe80::a00:27ff:fe54:36e7/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
roysanchez@roysanchez:~$
```

3) Máquina víctima (Configurar Suricata)

Vamos a decirle a Suricata qué red proteger, por dónde escuchar y qué reglas cargar. Sigue estos pasos en tu Ubuntu Server:

- Abrimos el archivo de configuración:

sudo nano /etc/suricata/suricata.yaml

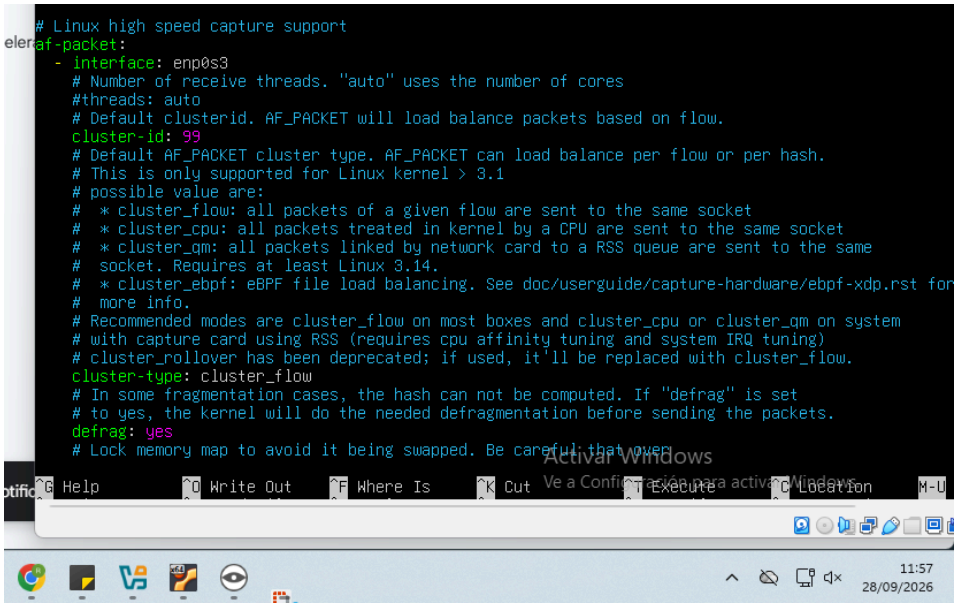


- En `vars:` bajo `address-groups:`, define tu red:

  HOME_NET: "[192.168.56.0/24]"

- En la sección `af-packet:`, pon tu interfaz interna:

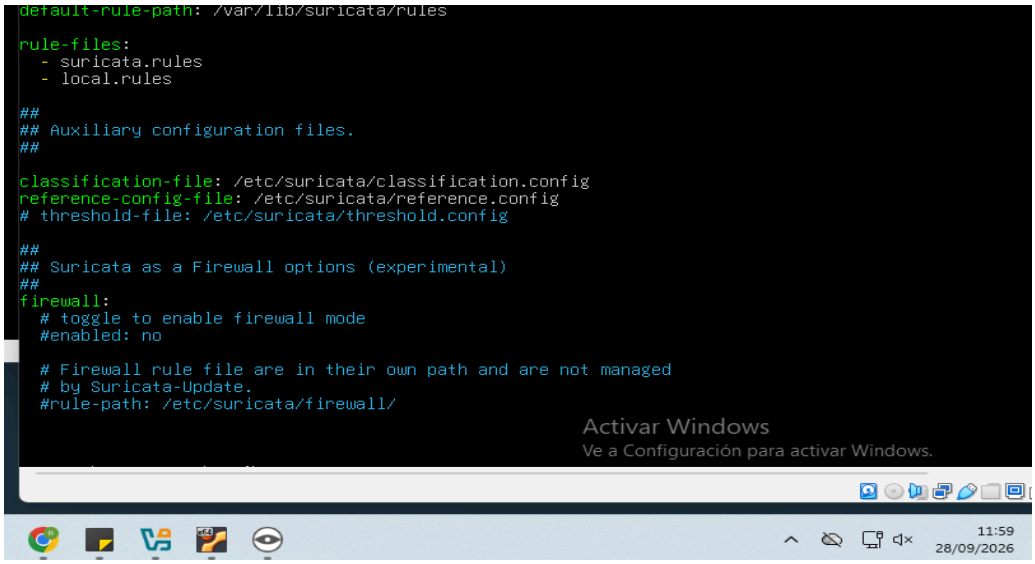
  interface: enp0s3



- En la sección `rule-files:`, asegúrate de que aparecen estas dos líneas (añade la de `local.rules`):

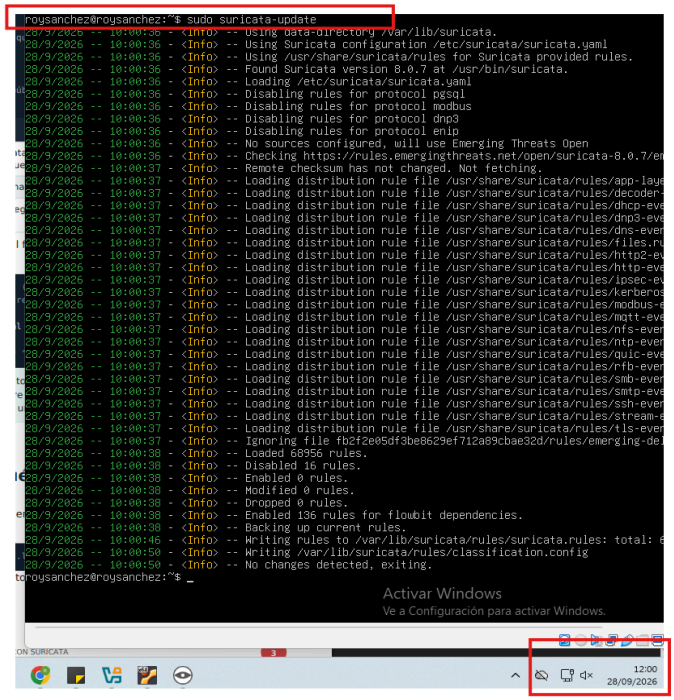
  - suricata.rules

  - local.rules



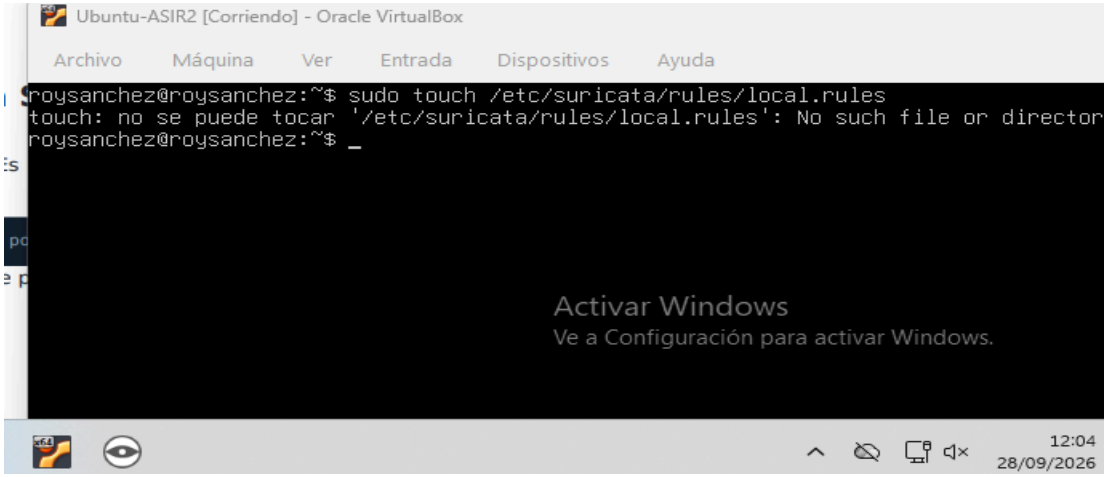
Ahora se descargan las reglas públicas, se crea el fichero de reglas propias y se comprueba que la configuración es válida antes de arrancar:

sudo suricata-update



Me aparece un error al ejecutar el siguiente comando:

sudo touch /etc/suricata/rules/local.rules



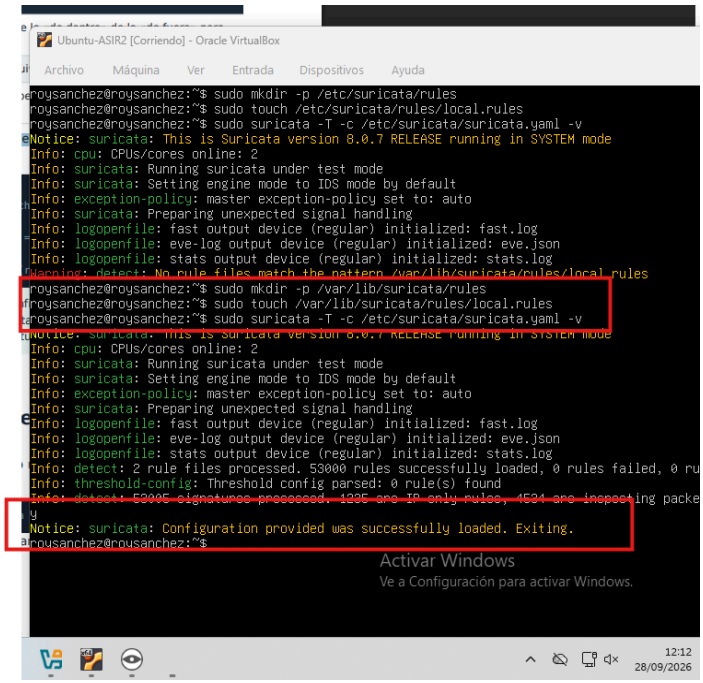
sudo mkdir -p /etc/suricata/rules

Crea el archivo vacío para tus reglas locales:

sudo touch /etc/suricata/rules/local.rules

Valida la configuración con el test (muy importante para asegurarnos de que no hay fallos de sintaxis en el `.yaml`):

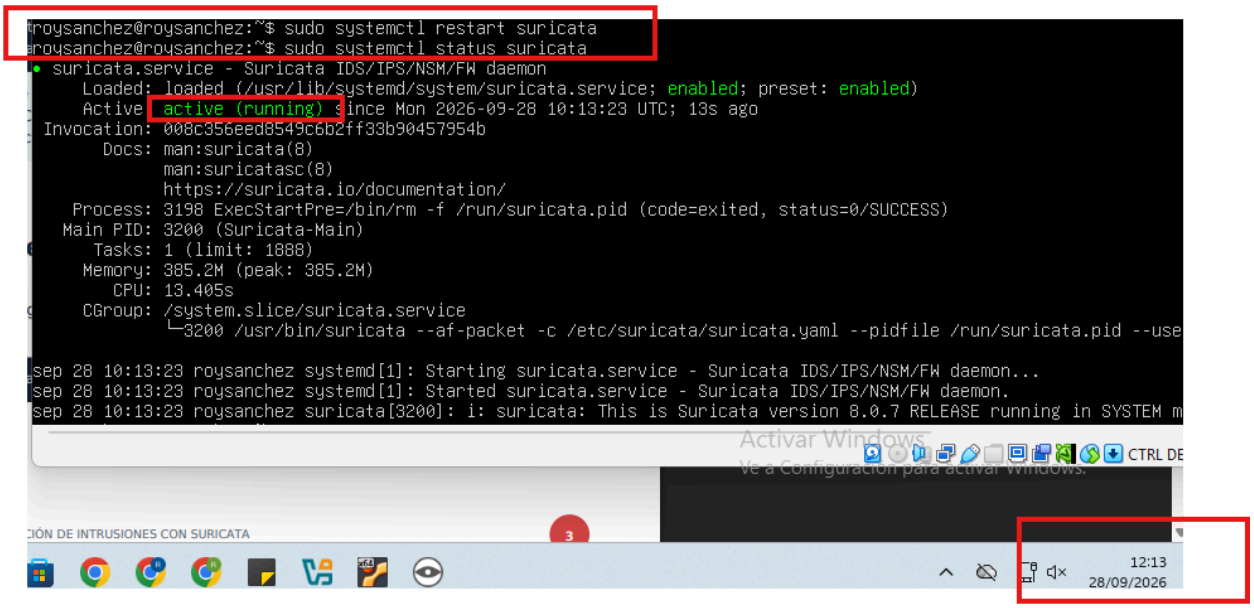
sudo suricata -T -c /etc/suricata/suricata.yaml -v



Verificamos con

sudo systemctl restart suricata

sudo systemctl status suricata



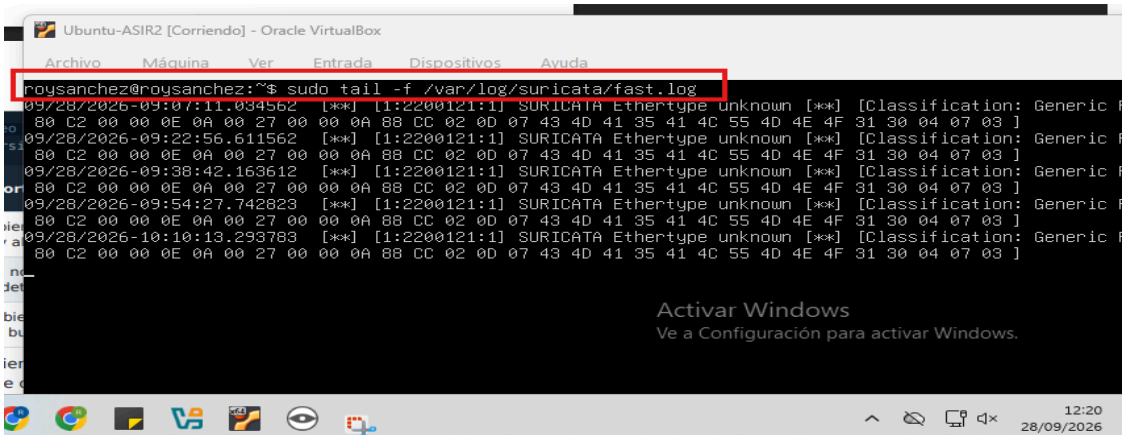
4) Atacar y observar (Provocar el ataque y ver qué detecta Suricata de fábrica).

4.1 Máquina víctima:

Para esta práctica vamos a usar las dos máquinas atacante y víctima a la vez.

Lo primero, preparar el Suricate en la víctima para que se genere las alter con el siguiente comando

sudo tail -f /var/log/suricata/fast.log

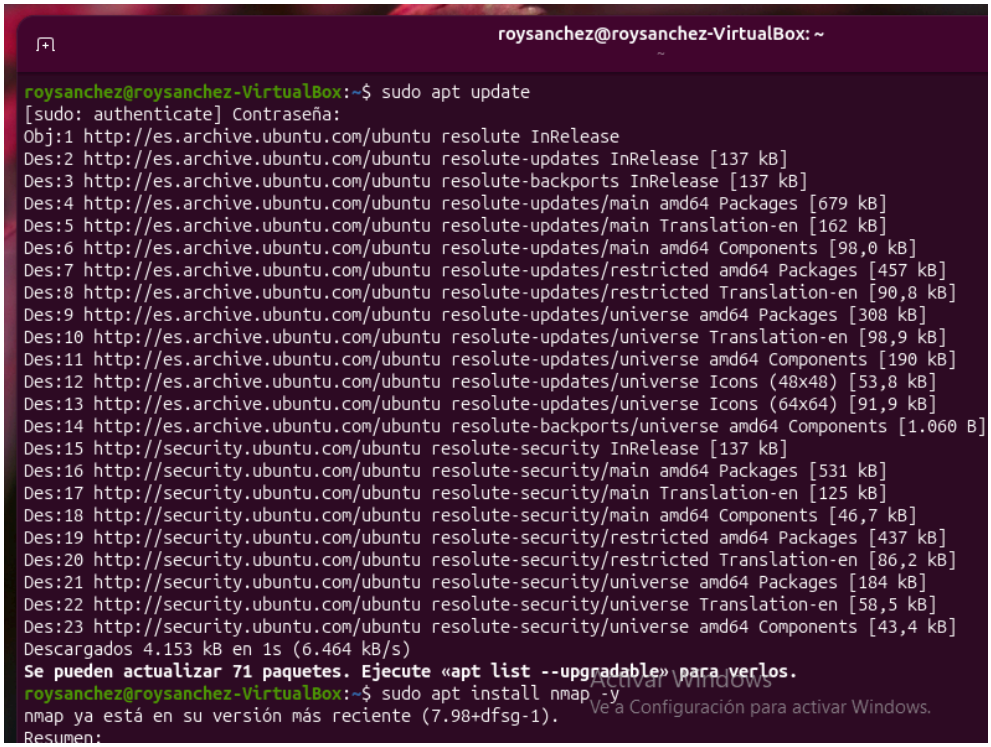


4.2 Máquina Atacante.

Lo primero es descargar Nmap

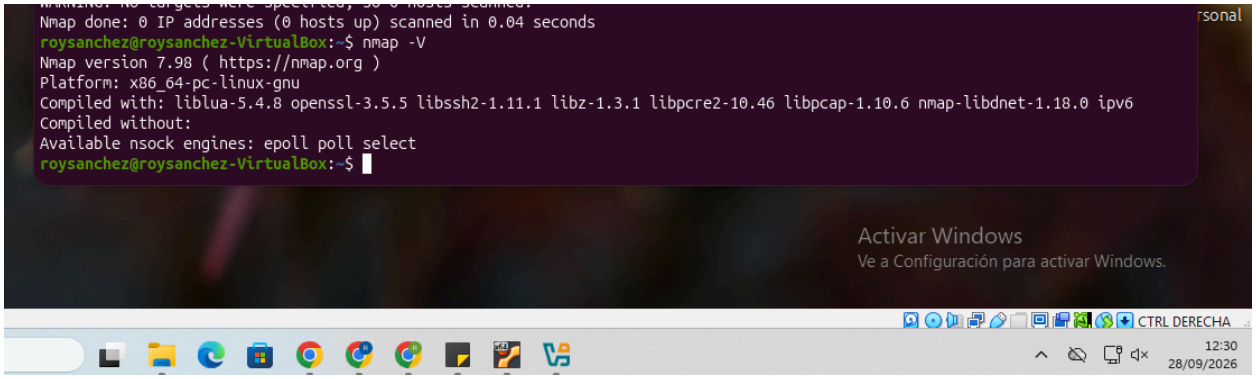
sudo apt update → para actualizar el sistemas

sudo apt install nmap -y → para instalar Nmap



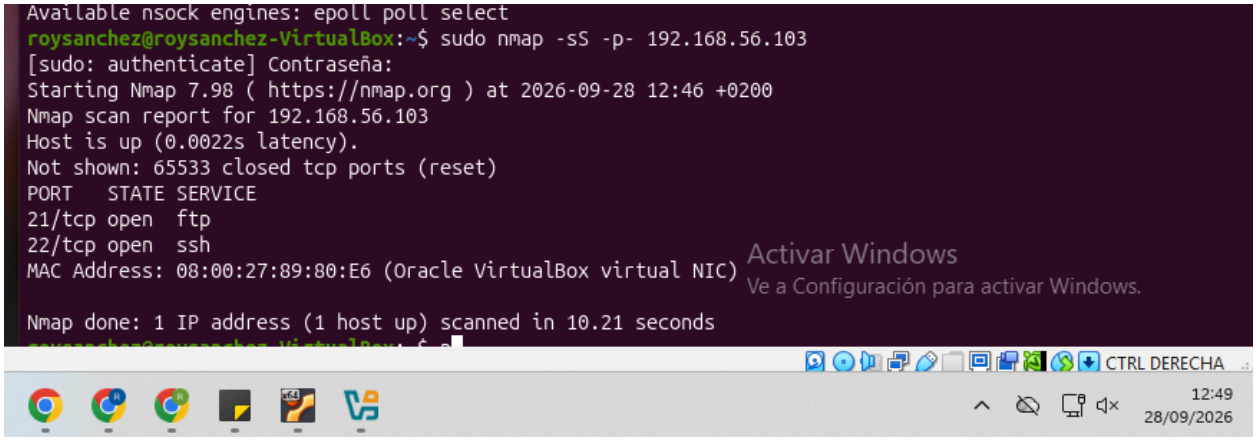
Confirmamos con el siguiente comando que se haya instalado correctamente.

nmap -V



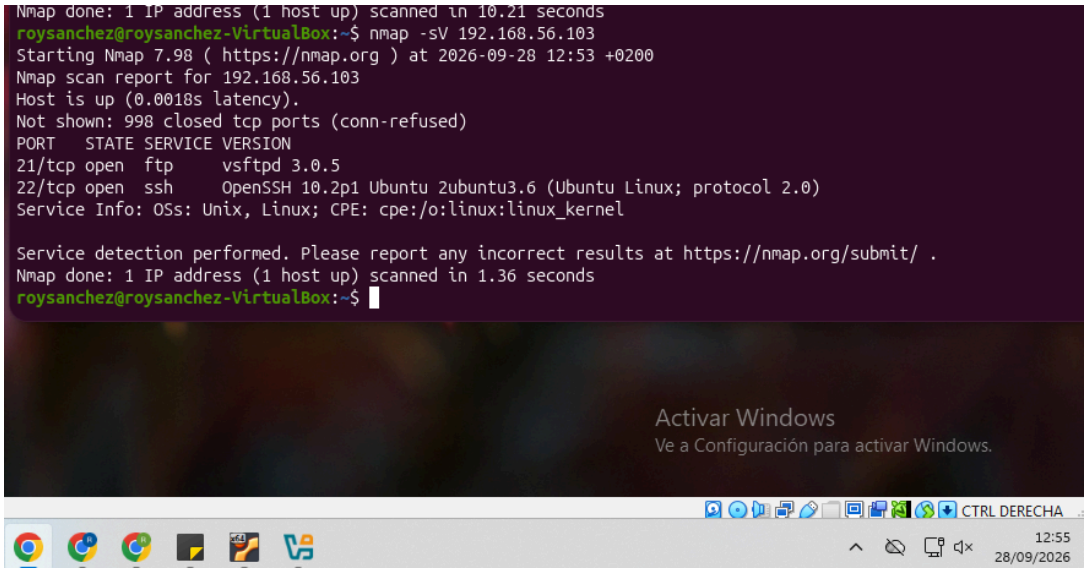
Desde el atacante, lanzamos la fase de reconocimiento: un escaneo de puertos para averiguar qué servicios ofrece la víctima.

sudo nmap -sS -p- 192.168.56.103



Escaneamos para ver las versiones de

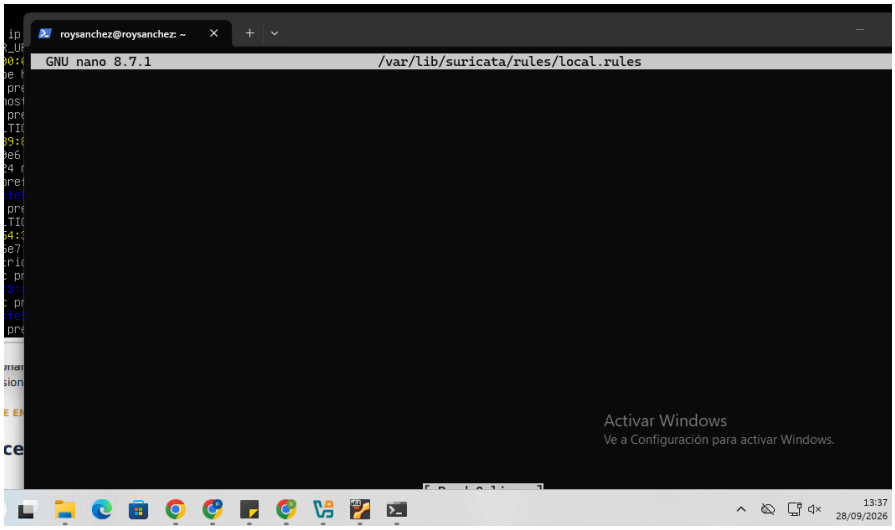
nmap -sV 192.168.56.103



Los escaneos de Nmap han funcionado a la perfección. Han detectado que los puertos 21 (ftp) y 22 (ssh) están abiertos en tu servidor y han identificado que el servicio en el puerto 21 es exactamente vsftpd 3.0.5, el servidor vulnerable que instalamos en el Paso 1.

Suricata no suele generar una alerta clara por defecto para este tipo de escaneo SYN porque requiere una regla específica para contar esos intentos en un marco de tiempo corto.

He habilitado un SSH desde el equipo anfitrión para copiar y pegar todo las reglas y que quede todo correcto.



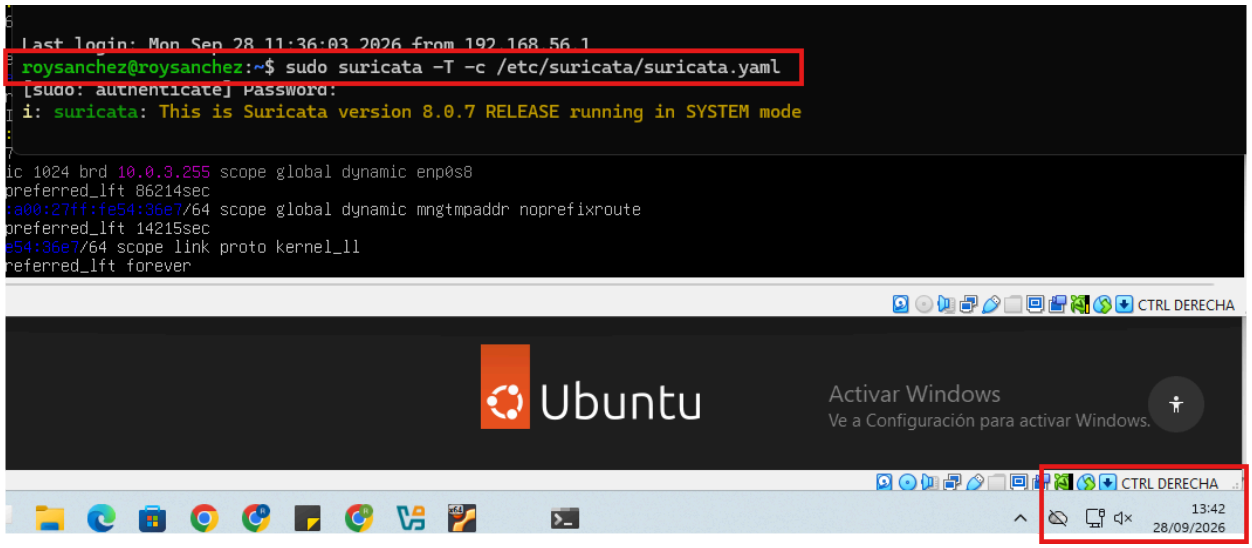
Una vez tengamos esto guardamos las siguientes reglas:

alert tcp any any -> $HOME_NET any (msg:"UT2 Posible escaneo SYN"; flags:S; threshold:type both, track by_src, count 20, seconds 5; classtype:attempted-recon; sid:1000001; rev:1;)

alert tcp any any -> $HOME_NET 21 (msg:"UT2 Credenciales FTP en claro"; flow:to_server,established; content:"PASS "; nocase; classtype:policy-violation; sid:1000002; rev:1;)

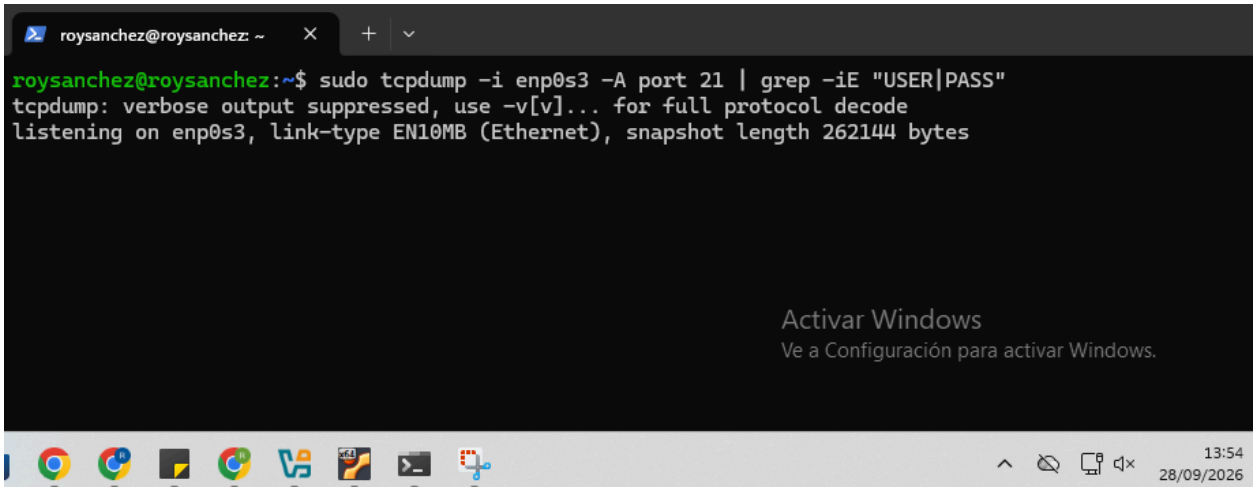
sudo suricata -T -c /etc/suricata/suricata.yaml

sudo systemctl restart suricata



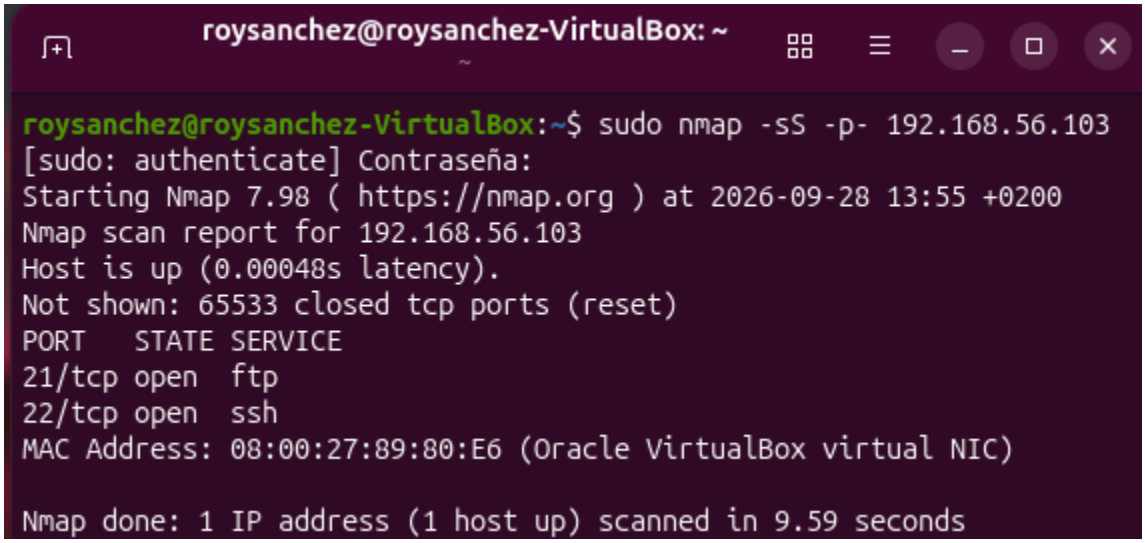
En tu Ubuntu Server (Máquina Víctima): Vamos a poner a `tcpdump` a "escuchar" el puerto 21 de tu tarjeta de red interna. lo hacemos con el siguiente comando:

sudo tcpdump -i enp0s3 -A port 21 | grep -iE "USER|PASS"

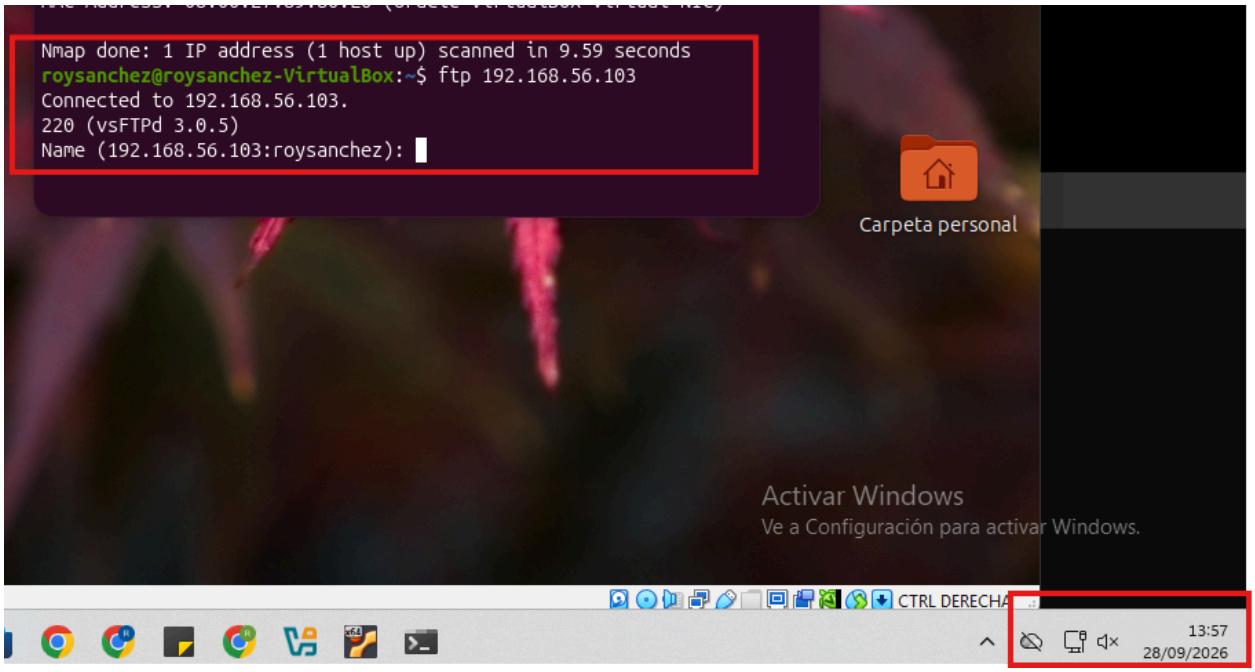


Lanza el escaneo sigiloso para disparar tu Regla 1

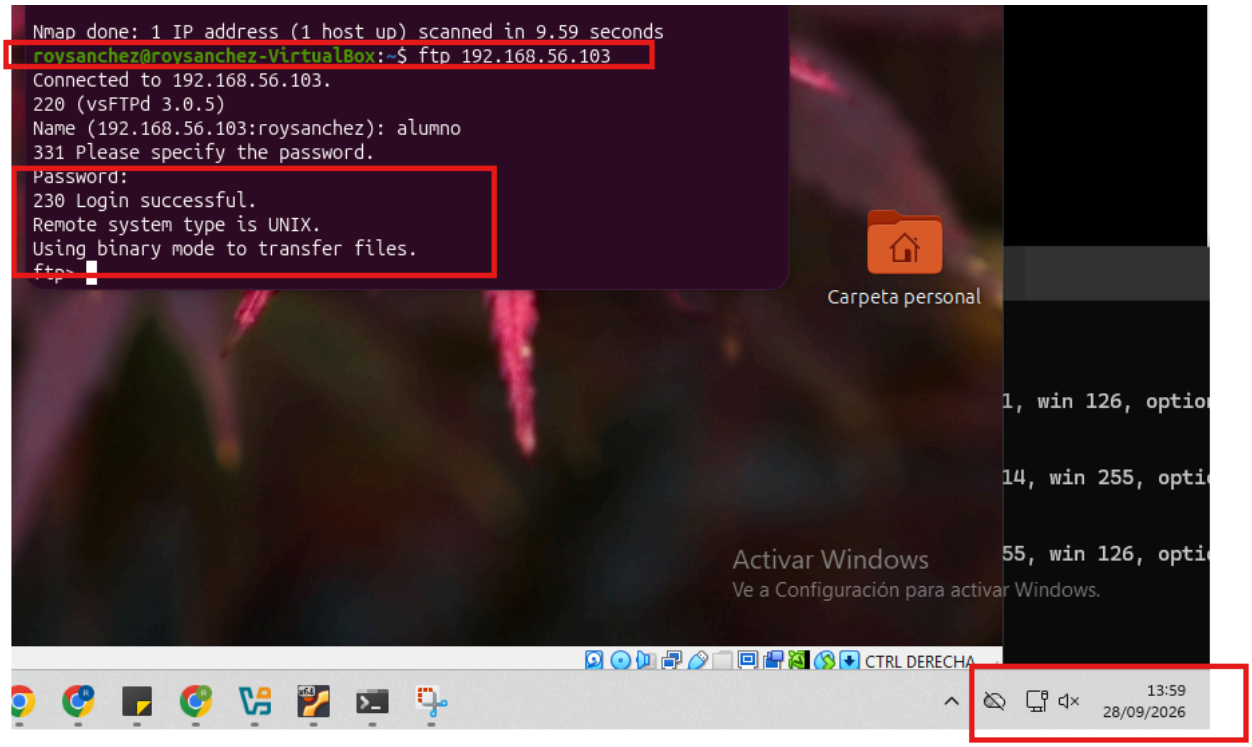
sudo nmap -sS -p- 192.168.56.103

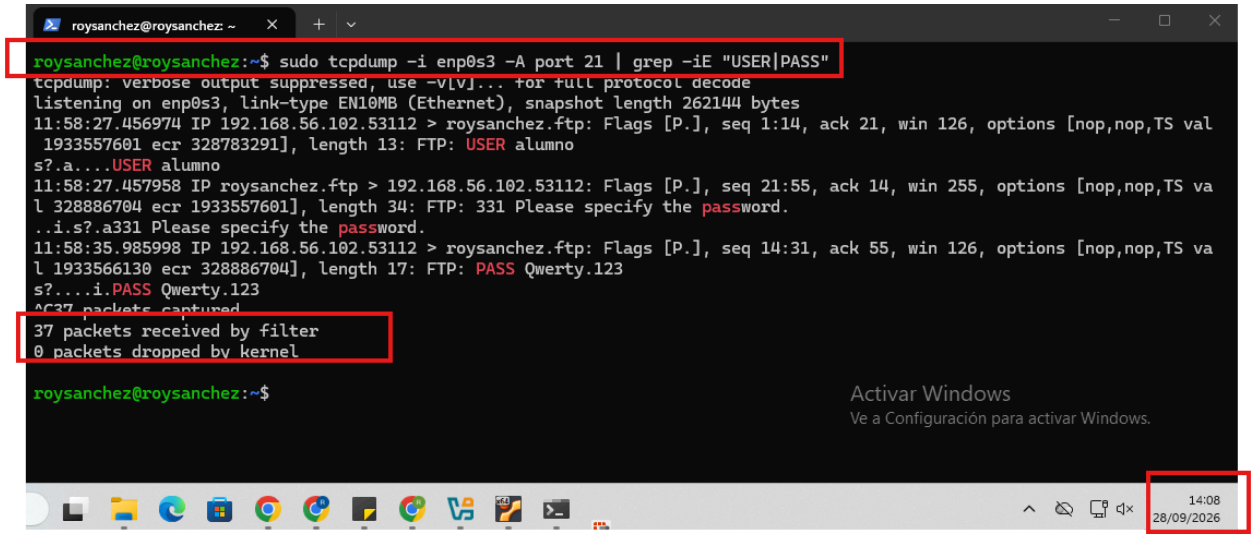


En cuanto se acabe el escaneo anterior, nos conectamos al servidor FTP para disparar la Regla 2



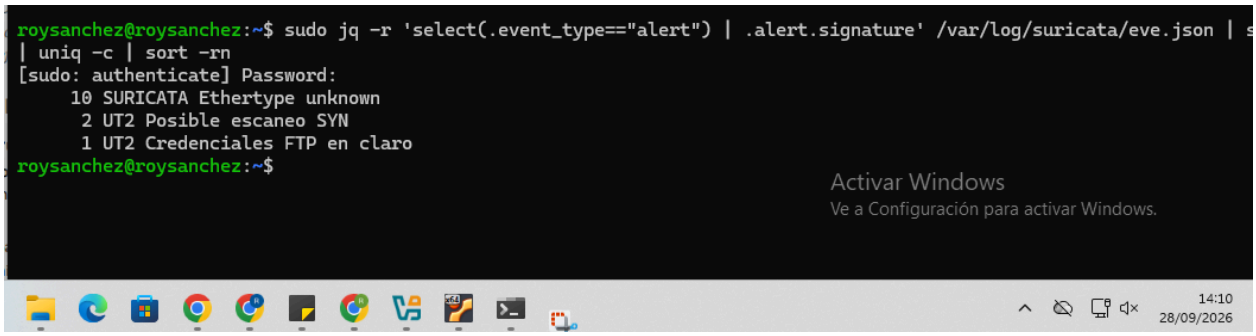
Accedemos con el usuario creado





Luego, Por último, ejecuto este comando para ver el recuento final de alertas que Suricata ha guardado en su registro JSON (funcionando como un SIEM):

sudo jq -r 'select(.event_type=="alert") | .alert.signature' /var/log/suricata/eve.json | sort | uniq -c | sort -rn

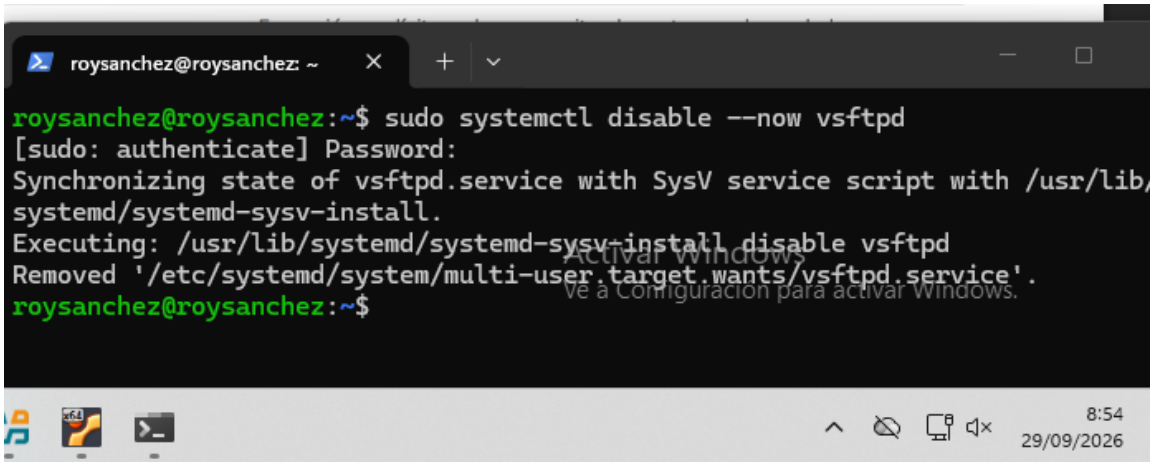


5) Contramedida

Como dice el enunciado, detectar es la mitad del trabajo; la otra mitad es cerrar la puerta. El apartado 2.3 lo dice claro: cada servicio expuesto es una vía de entrada. Apagamos el FTP inseguro y levantamos un cortafuegos que deniega por defecto y solo abre lo imprescindible.

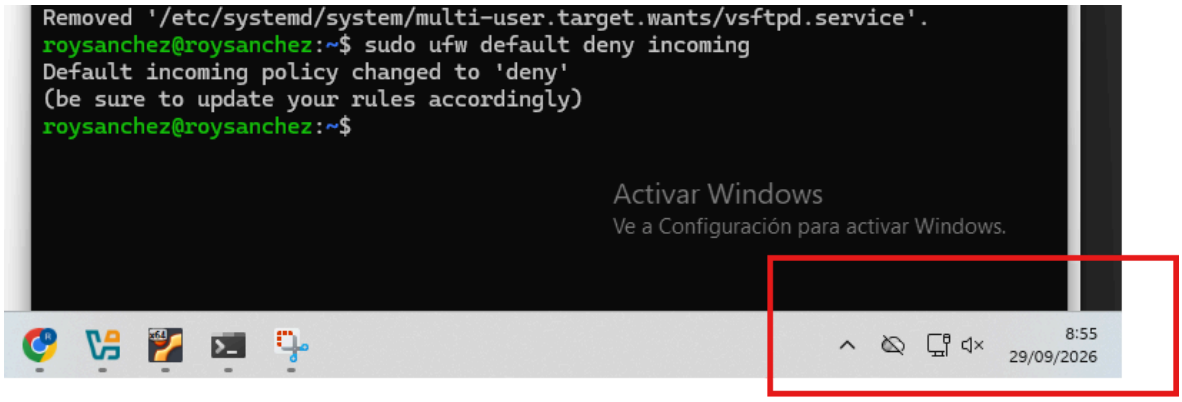
5.1 Apagamos y deshabilitamos el servicio FTP para que no vuelva a arrancar:

con el siguiente comando: sudo systemctl disable --now vsftpd



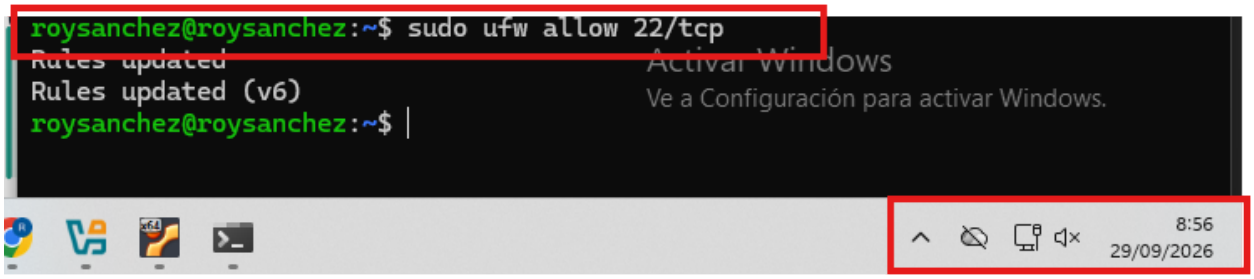
5.2) Configurar la política base del cortafuegos para denegar todo el tráfico entrante:

sudo ufw default deny incoming



5.3) Abrimos una excepción explícita solo para el puerto SSH (el único que realmente necesitamos para administrar):

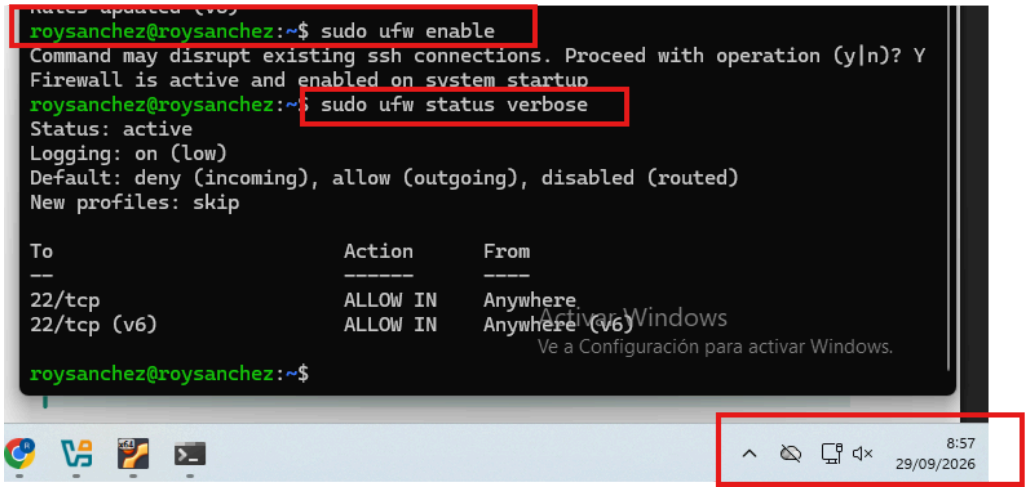
sudo ufw allow 22/tcp



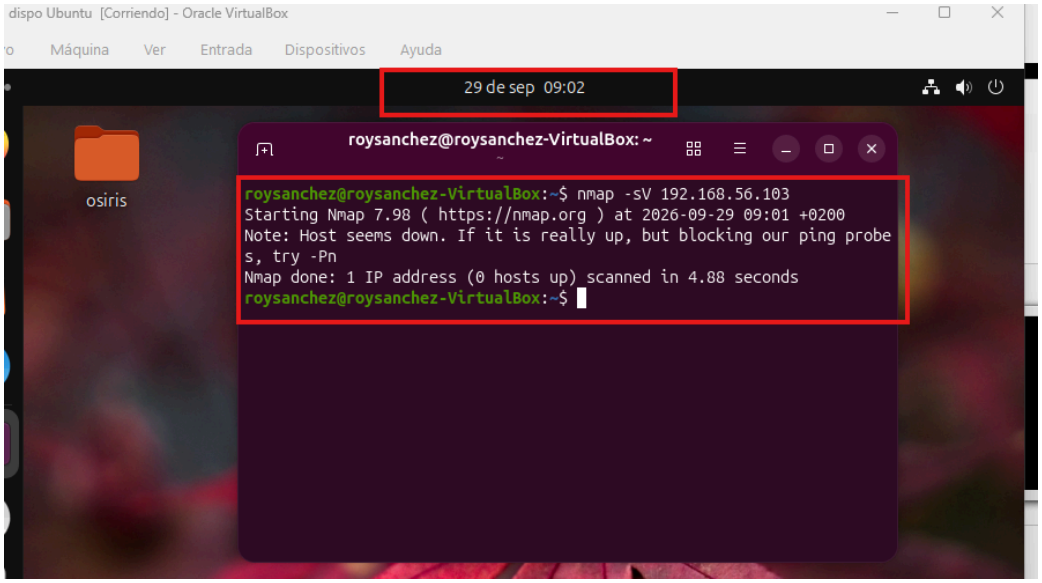
5.4) Activamos el cortafuegos y comprueba cómo ha quedado la configuración:

sudo ufw enable

sudo ufw status verbose



5.5) **Verificación final vamos a la máquina atacante (Ubuntu Usuario)** y lanzamos de nuevo el escaneo de versiones para comprobar que la superficie de ataque se ha reducido:



En la captura de pantalla de arriba el resultado del escaneo es la comprobación que hemos lanzado desde la máquina atacante tras haber implementado las contramedidas en el servidor. Al ejecutar el comando nmap -sV 192.168.56.103, el Nmap interrumpe el análisis y devuelve el mensaje de “Host Seems down”. Este comportamiento demuestra el éxito de la política de cortafuegos UFW que he configurado. Al denegar por defecto to el tráfico entrante, por este motivo el servidor descarta de forma silenciosa las sondas previas de descubrimiento que la Nmap envía para comprobar si el puerto está activo. Como no recibe respuesta Nmap entiende que está apagada y aborta el escaneo de puertos. Con esto se puede reducir drásticamente la superficie de ataque. no solo he eliminado el servicio DTP vulnerable, sino que el servidor ya no responde a tanteos básicos de reconocimientos, obligando a un posible atacante a realizar escaneos a ciegas forzando el parámetro -Pn y dificultando significativamente la fase inicial de descubrimiento de red.

Cuestionarios

1) Si cambiaras el umbral de la Regla 1 de count 20 a count 2, ¿qué pasaría con el número de alertas? Relaciónalo con «el cálculo de los falsos positivos que hunde los proyectos» (2.5): ¿por qué un umbral demasiado bajo hace menos segura la red y no más?

Respondiendo a esta pregunta:

Si reducimos el umbral a count 2 en 5 segundos, el número de aletas se dispararía. Normalmente el usuario establece conexiones legítimas (Por ejemplo, cargando una página web que abre múltiples flujos simultáneos) Ejemplo, podría generar fácil dos paquetes SYN seguidos. Esto provocaría una avalancha de falsos positivos. Como se indica en la teoría, un umbral demasiado bajo genera fatiga de alertas en los administradores. Esto haría que la red sea menos segura ya que los atacantes reales acaban siendo ignorados o se camuflan entre miles de alertas falsas.

2) La Regla 2 no detectaría nada si la empresa usara SFTP en lugar de FTP. ¿Por qué? ¿Qué principio de la tríada CIA protege el cifrado del transporte?

Respondiendo a esta pregunta:

En este caso la regla 2 no detectaría el ataque porque SFTP cifra toda la comunicación de extremo a extremo. Suricata analiza el tráfico buscando la cadena literal de texto (PASS) en claro; al estar el paquete cifrado el IDS solo lee una cadena de caracteres ilegibles y la regla no encuentra coincidencia. Por ende, el cifrado protege directamente el principio de confidencialidad dentro de la tríada CIA; ya que al estar cifrada impide que terceros intercepten y se lean los datos.

3) ¿Qué tendrías que cambiar para que Suricata bloquee el ataque en vez de solo avisar (pasar de IDS a IPS)? Nombra la acción de la regla y el modo en que debe correr Suricata.

Respondiendo a esta pregunta:

Para que Suricata pase a bloquear activamente el tráfico malicioso

- En la regla de detección: Se debe cambiar la acción de alert por drop que significa que descarta el paquete silenciosamente o reject que sería corta la conexión informando al emisor.
- En la ejecución del servicio: Suricata debe configurarse para correr en modo Inline (IPS), por ejemplo, utilizando NFQ Netfilter Queue o configurando AF_PACKET en modo IPS. Así el tráfico pasa físicamente a través del motor y puede ser detenido antes de llegar a su destino.

4) Sitúa cada una de tus dos reglas en la tabla de fases del ataque (1.8): ¿en qué fase actúa la del escaneo SYN y en qué fase la de las credenciales FTP?

Respondiendo a esta pregunta:

- **Regla 1 (Escaneo SYN):** Actúa en la fase de **Reconocimiento**. El atacante aún no ha entrado al sistema; está tanteando la red para descubrir puertos abiertos y mapear la superficie de ataque.
- **Regla 2 (Credenciales FTP):** Actúa en la fase de **Explotación / Robo de credenciales**. El atacante ya interactúa con un servicio inseguro para obtener un acceso no autorizado al sistema.

5) El apartado 2.5 pide dimensionar el almacenamiento. Si tu Suricata generase 500 eventos/segundo y cada evento ocupara 500 bytes, ¿cuántos GB al día produciría? ¿Y con 30 días de retención?

Respondiendo a esta pregunta:

Volumen por segundo: 500 eventos/s × 500 bytes/evento = 250.000 bytes/s.

Segundos en un día: 60 segundos × 60 minutos × 24 horas = 86.400 segundos.

Bytes totales al día: 250.000 bytes/s × 86.400 s = 21.600.000.000 bytes.

Conversión (dividiendo entre 10^9): **21,6 GB diarios.**

Cálculo para 30 días de retención:

21,6 GB/día × 30 días = 648 GB de almacenamiento necesario en el SIEM.

6) ¿Por qué se recomienda ejecutar siempre suricata -T antes de reiniciar el servicio? ¿Qué relación tiene con «probar en pre-producción antes de aplicar» de la gestión de parches (1.4)?

Se recomienda ejecutar siempre suricata -T porque valida la sintaxis completa de la configuración y las reglas. Un solo error tipográfico, como un punto y coma faltante, provocaría que el servicio Suricata aborte su arranque tras un reinicio. La relación con la "prueba en pre-producción" es directa: si se aplica un cambio sin validarlo y este falla en producción, se deja a la red completamente desprotegida; probar antes garantiza la continuidad y disponibilidad del sistema defensivo.

Conclusión

Como cierre esta práctica me ha demostrado qué tan expuestos están nuestros entornos cuando no aplicamos medidas de seguridad activa desde la propia administración de la red. Ver con mis propios ojos cómo una contraseña se desplaza en texto claro y es interceptada mediante tcpdump es una prueba definitiva que confirma por qué ciertos protocolos obsoletos son un riesgo real. Además, he comprobado que un motor potente como Suricata es ciego si no lo afinamos con las reglas correctas, y que la verdadera protección nunca depende de un único software, sino de combinar la inteligencia de un IDS con políticas de cortafuegos estrictas que bloqueen todo tráfico no esencial por defecto.